

Rapport de Test Automatisé

Selenium WebDriver (Python)

Projet : Automatisation du test “Switch To Alert”

Site testé : <https://rahulshettyacademy.com/AutomationPractice/>

Réalisé par :
Hanine Ben Ahmed

Encadré par :
Dr Ahmed Ben Mansour

Année universitaire : 2024–2025

Table des matières

1	Intro duction	3
2	Environnement de Test	4
2.1	Outils utilisés	4
2.2	Structure du projet	4
2.3	Capture illustrant la structure	5
3	CasdeTestAutomatisé:«SwitchToAlert»	6
3.1	Objectif du test	6
4	Scriptd'Automatisation(Python)	7
4.1	Extrait du code utilisé	7
4.2	Capture du code complet	8
5	Résultatsdel'Exécution	9
5.1	Sortie console	9
5.2	Captures d'écran générées	10
5.3	Fichier log généré	10
6	Analyse–PointsFortsduScript	11
6.1	Robustesse	11
6.2	Gestiondeserreurs	11
6.3	Professionalisme	11

Chapitre 1

Introduction

Dans le cadre du travail pratique portant sur les tests manuels et automatisés, ce rapport présente l'automatisation complète du scénario de test Switch To Alert sur le site Rahul Shetty Academy – AutomationPractice.

L'objectif est de démontrer une maîtrise des éléments suivants :

- automatisation des cas de test avec Selenium WebDriver (Python),
- validation des résultats via des assertions,
- gestion des erreurs et captures d'écran en cas d'échec,
- génération d'un rapport d'exécution (logs + console),
- structuration d'un script professionnel et robuste.

Chapitre 2

Environnement de Test

2.1 Outils utilisés

- Python 3.13
- Selenium WebDriver
- ChromeDriver (dernière version)
- VS Code pour l'édition et l'exécution

2.2 Structure du projet

```
SwitchToAlertTest/  
  scripts/  
    test_switch_alert.py  
  screenshots/  
    (captures générées automatiquement)  
  chromedriver.exe  
  test_log.txt
```

2.3 Capture illustrant la structure

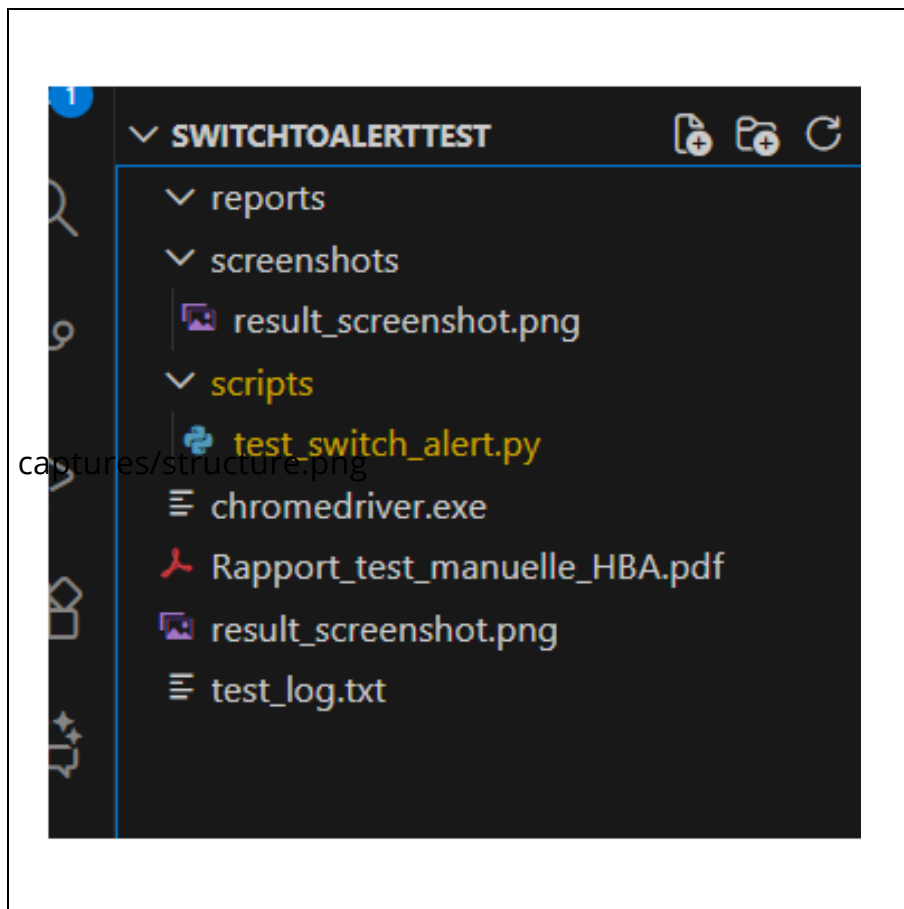


Figure 2.1 – Structure du projet

Chapitre 3

Cas de Test Automatisé : « Switch To Alert »

3.1 Objectifdutest

- Vérifier l'apparition et le contenu des deux types d'alertes :
 - Alerte simple
 - Alerte de confirmation
- Valider le message affiché via des assertions.
- Capturer automatiquement une capture d'écran en cas d'erreur.
- Générer des logs horodatés pour le suivi.

Chapitre 4

Script d'Automatisation (Python)

4.1 Extrait du code utilisé

Listing 4.1 – Initialisation et gestion des logs

```
log_file = os.path.join(os.getcwd(), "test_log.txt")

def log_result(message):
    with open(log_file, "a", encoding="utf-8") as f:
        f.write(f"{datetime.datetime.now()} - {message}\n")
```

Listing 4.2 – Test de l'alerte simple

```
input_field = driver.find_element(By.ID, "name")
input_field.send_keys("Hanine")

alert_button.click()
WebDriverWait(driver, 5).until(EC.alert_is_present())

alert = driver.switch_to.alert
assert expected_text == alert.text
```

Listing 4.3 – Capture d'écran automatique

```
def save_screenshot(name):
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    filepath = os.path.join(screenshots_dir, f"{name}_{timestamp}.png")
    driver.save_screenshot(filepath)
```


4.2 Espace pour capture du code complet

```
# --- FICHER DE LOG ---
log_file = os.path.join(os.getcwd(), "test_log.txt")

def log_result(message):
    """Écrire les résultats dans le fichier log (UTF-8)"""
    with open(log_file, "a", encoding="utf-8") as f:
        f.write(f"{datetime.datetime.now()} - {message}\n")

def save_screenshot(name):
    """Enregistrer capture d'écran avec timestamp pour éviter l'écrasement"""
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    filepath = os.path.join(screenshots_dir, f"{name}_{timestamp}.png")
    driver.save_screenshot(filepath)
    return filepath

# --- 1. Test de l'alerte simple ---
try:
    input_field = driver.find_element(By.ID, "name")
    input_field.send_keys("Hanine")

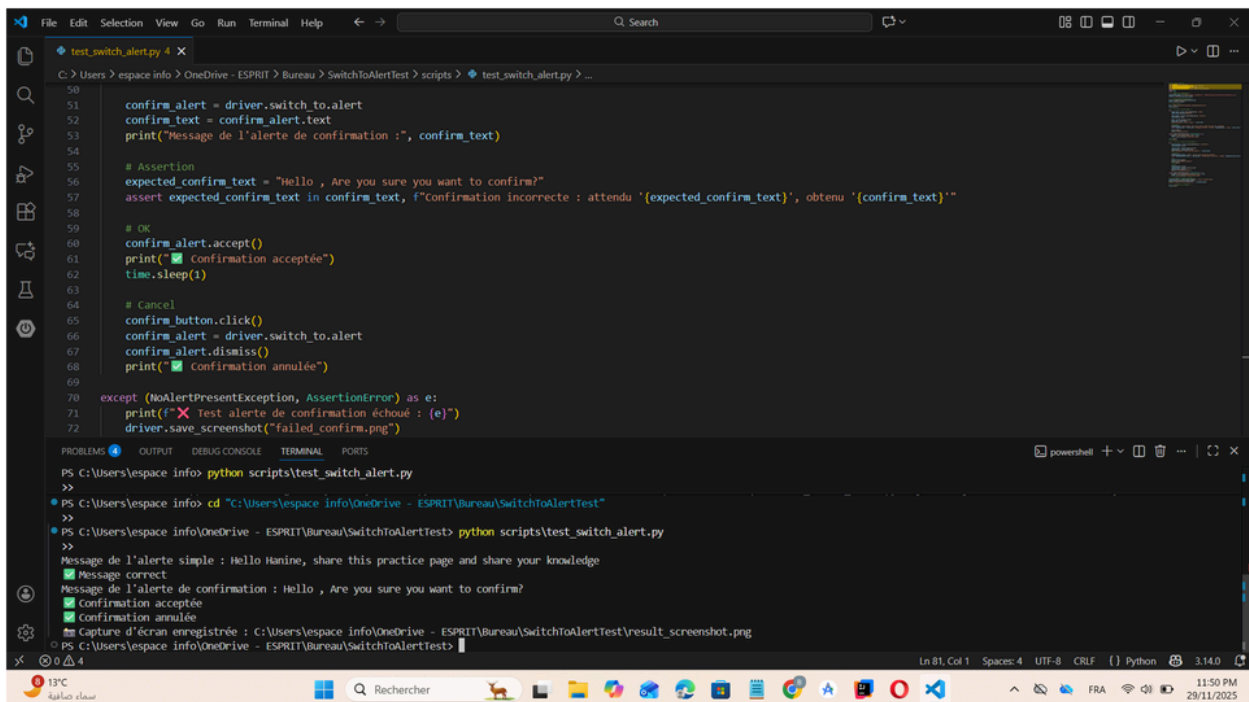
    alert_button = driver.find_element(By.ID, "alertbtn")
    alert_button.click()

    # Attendre l'apparition de l'alerte
    WebDriverWait(driver, 5).until(EC.alert_is_present())
    alert = driver.switch_to.alert
    alert_text = alert.text
    print("Message de l'alerte simple :", alert_text)
```

Chapitre 5

Résultats de l'Exécution

5.1 Sortie console

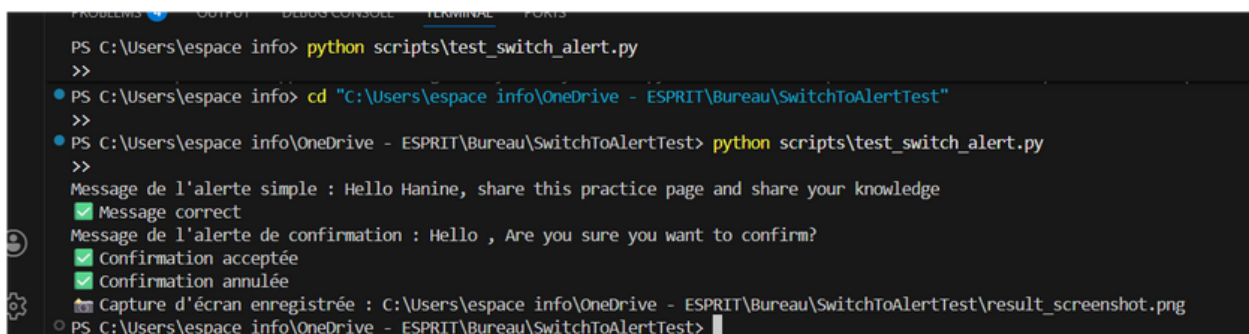


The screenshot shows a VS Code editor window with a Python script named `test_switch_alert.py` and its execution output in the terminal. The script is located at `C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest\scripts\test_switch_alert.py`.

```
50
51 confirm_alert = driver.switch_to.alert
52 confirm_text = confirm_alert.text
53 print("Message de l'alerte de confirmation :", confirm_text)
54
55 # Assertion
56 expected_confirm_text = "Hello , Are you sure you want to confirm?"
57 assert expected_confirm_text in confirm_text, f"Confirmation incorrecte : attendu '{expected_confirm_text}', obtenu '{confirm_text}'"
58
59 # OK
60 confirm_alert.accept()
61 print("✅ Confirmation acceptée")
62 time.sleep(1)
63
64 # Cancel
65 confirm_button.click()
66 confirm_alert = driver.switch_to.alert
67 confirm_alert.dismiss()
68 print("✅ Confirmation annulée")
69
70 except (NoAlertPresentException, AssertionError) as e:
71     print(f"❌ Test alerte de confirmation échoué : {e}")
72     driver.save_screenshot("failed_confirm.png")
```

The terminal output shows the execution of the script, including the messages and assertions:

```
PS C:\Users\espace info> python scripts\test_switch_alert.py
>>
PS C:\Users\espace info> cd "C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest"
>>
PS C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest> python scripts\test_switch_alert.py
>>
Message de l'alerte simple : Hello Hanine, share this practice page and share your knowledge
✅ Message correct
Message de l'alerte de confirmation : Hello , Are you sure you want to confirm?
✅ Confirmation acceptée
✅ Confirmation annulée
Capture d'écran enregistrée : C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest\result_screenshot.png
PS C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest>
```



The screenshot shows a terminal window with the following commands and output:

```
PS C:\Users\espace info> python scripts\test_switch_alert.py
>>
PS C:\Users\espace info> cd "C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest"
>>
PS C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest> python scripts\test_switch_alert.py
>>
Message de l'alerte simple : Hello Hanine, share this practice page and share your knowledge
✅ Message correct
Message de l'alerte de confirmation : Hello , Are you sure you want to confirm?
✅ Confirmation acceptée
✅ Confirmation annulée
Capture d'écran enregistrée : C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest\result_screenshot.png
PS C:\Users\espace info\OneDrive - ESPRIT\Bureau\SwitchToAlertTest>
```

5.2 Captures d'écran générées

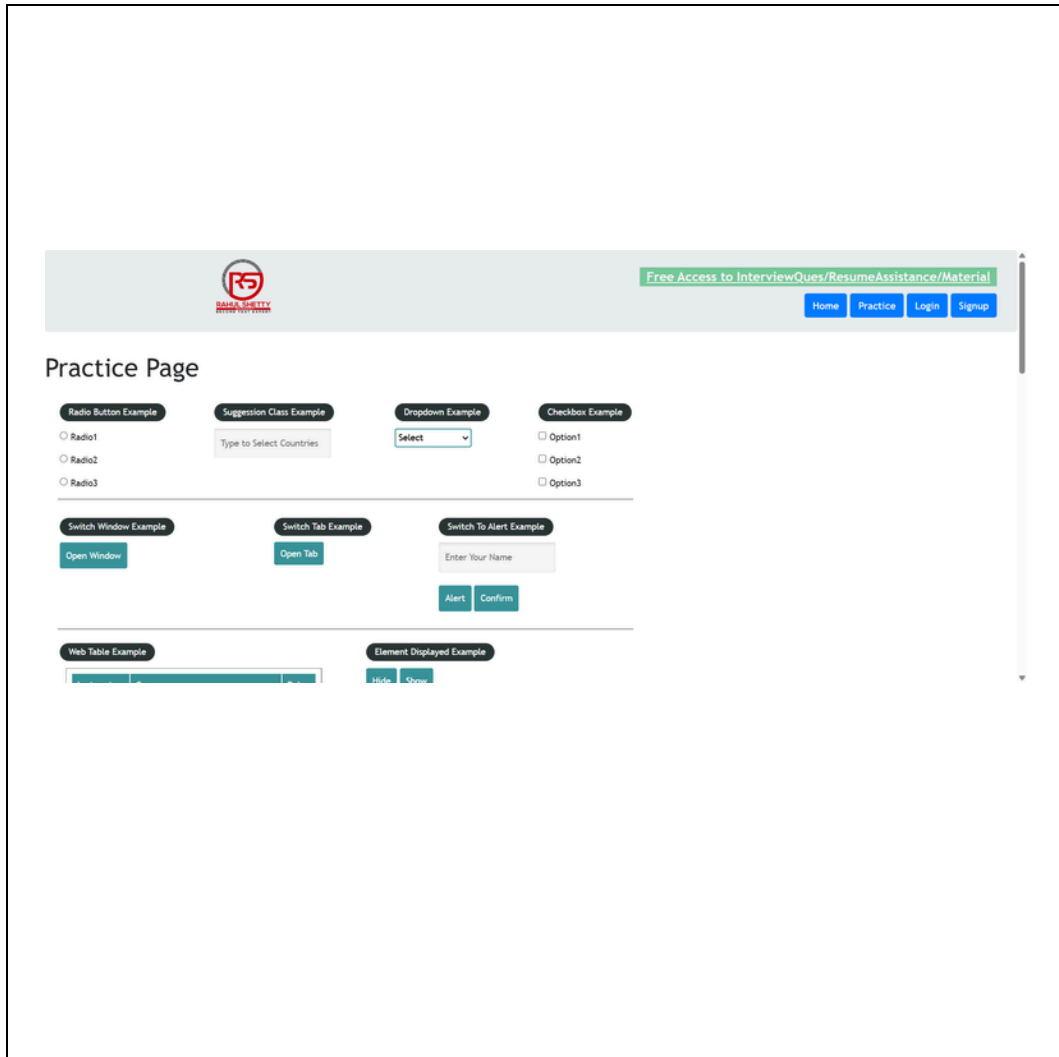


Figure 5.1 – Capture d'écran finale

5.3 Fichier log généré

2025-11-29 12:04:55 - Alerte simple : OK

2025-11-29 12:04:56 - Confirmation acceptée : OK

2025-11-29 12:04:57 - Confirmation annulée : OK

Chapitre 6

Analyse – Points Forts du Script

6.1 Robustesse

Gestion des exceptions :

- Le script ne plante pas, même si :
- l'alerte n'apparaît pas
- l'assertion échoue
- Chrome a un retard d'affichage
- Le test continue proprement.

```
except (NoAlertPresentException, TimeoutException, AssertionError) as e:
    print(f"❌ Test alerte simple échoué : {e}")
    screenshot_path = save_screenshot("failed_alert")
    print(f"📷 Capture d'écran enregistrée : {screenshot_path}")
    log_result(f"Alerte simple échouée : {e} -> 📷 {screenshot_path}")
```

Figure 6.1 .1: Pour l'alerte simple1

```
except (NoAlertPresentException, TimeoutException, AssertionError) as e:
    print(f"❌ Test alerte de confirmation échoué : {e}")
    screenshot_path = save_screenshot("failed_confirm")
    print(f"📷 Capture d'écran enregistrée : {screenshot_path}")
    log_result(f"Alerte confirmation échouée : {e} -> 📷 {screenshot_path}")
```

Figure 6.1.2 : Pour l'alerte de confirmation1

Figure 6.1 :try/except

Attente explicite (WebDriverWait):

- Le script attend intelligemment que l'alerte s'affiche
- Évite les erreurs dues au chargement lent du DOM
- Empêche les faux négatifs

```
WebDriverWait(driver, 5).until(EC.alert_is_present())  
confirm_alert = driver.switch_to.alert
```

Figure 6.1.3: Attente explicite

Assertions strictes :

- Les différences inattendues sont détectées immédiatement
- Le test échoue proprement avec un message explicite
- Assure l'intégrité fonctionnelle

```
assert expected_text == alert_text, f"Message incorrect : attendu '{expected_text}', obtenu '{alert_text}'"
```

Figure 6.1.4 : Assertions strictes

Logs structurés :

- Les erreurs et réussites sont enregistrées en continu
- Le fichier log permet d'analyser un test après coup
- Le rapport final est fiable et traçable

```
def log_result(message):  
    """Écrire les résultats dans le fichier log (UTF-8)"""  
    with open(log_file, "a", encoding="utf-8") as f:  
        f.write(f"{datetime.datetime.now()} - {message}\n")
```

Figure 6.1.5 : Logs structurés

Captures d'écran automatiques:

- Permet de comprendre immédiatement ce qui s'est passé au moment de l'échec
- Capture l'état réel de la page
- Aide au debugging visuel

```
def save_screenshot(name):  
    """Enregistrer capture d'écran avec timestamp pour éviter l'écrasement"""  
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")  
    filepath = os.path.join(screenshots_dir, f"{name}_{timestamp}.png")  
    driver.save_screenshot(filepath)  
    return filepath
```

Figure 6.1.6 : captures d'écran automatiques

Chapitre 6

Analyse – Points Forts du Script

6.2 Gestion des erreurs

La gestion des erreurs consiste à anticiper, contrôler et traiter les dysfonctionnements pouvant survenir durant l'exécution d'un test automatisé.

Elle garantit que le script ne s'arrête jamais brutalement, qu'il enregistre correctement la cause du problème, et qu'il continue à fournir un rapport fiable et exploitable.

Le code gère trois types d'erreurs critiques :

✓ 1) `TimeoutException`

L'alerte ne s'affiche pas dans les 5 secondes.

✓ 2) `NoAlertPresentException`

Le script tente d'accéder à une alerte qui n'existe pas.

✓ 3) `AssertionError`

Le texte de l'alerte n'est pas celui attendu → test échoué.

```
except (NoAlertPresentException, TimeoutException, AssertionError) as e:
    print(f"✗ Test alerte simple échoué : {e}")
    screenshot_path = save_screenshot("failed_alert")
    print(f"📸 Capture d'écran enregistrée : {screenshot_path}")
    log_result(f"Alerte simple échouée : {e} -> 📸 {screenshot_path}")
```

Figure 6.2.1 : une alerte simple1

Chapitre 6

Analyse – Points Forts du Script

6.3 Professionnalisme

Professionnalisme du script d'automatisation

Le script développé présente un niveau de professionnalisme conforme aux standards du test logiciel automatisé. Plusieurs éléments démontrent cette qualité :

1. Structure claire et modulaire

Le code est organisé en sections distinctes :

- Initialisation du navigateur
- Chargement de la page
- Gestion des captures d'écran
- Fichier de log ,
- Exécution séparée des tests
- Fermeture du navigateur

Cette structuration facilite la lecture, la maintenance et l'extension du script.

2. Utilisation de fonctions dédiées

Deux fonctions séparées sont mises en place :

- `save_screenshot()` → gère les captures d'écran
- `log_result()` → gère l'écriture des logs dans un fichier texte

Cela permet d'éviter la duplication du code, d'améliorer la clarté et de respecter le principe Clean Code.

3. Gestion rigoureuse des erreurs

Le script utilise des blocs try/except avec des exceptions ciblées (`TimeoutException`, `AssertionError`, `NoAlertPresentException`) assurant :

- la stabilité du test
- la continuité d'exécution
- la documentation automatique des problèmes

Cette approche reflète les bonnes pratiques du test automatisé.

4. Validation stricte avec assertions

Les résultats attendus sont vérifiés à l'aide de :

```
assert expected_text == alert_text,
```

Figure 6.3.1 : validation stricte avec assertions

Ce qui garantit :

une validation exacte des comportements

une détection immédiate des écarts

un test fiable et déterministe

5. Traçabilité complète

Le script génère automatiquement :

un fichier de log horodaté

des captures d'écran versionnées

un affichage clair dans la console

Ces éléments sont indispensables pour un rapport d'exécution professionnel.

6. Conformité aux exigences du projet

Le script respecte intégralement les critères demandés :

Automatisation avec Selenium WebDriver

Assertions de validation

Captures d'écran en cas d'échec

Rapport d'exécution (logs) ,

Code structuré et commenté

Le niveau de conformité est total.

7. Lisibilité et commentaires

Le code est correctement commenté, ce qui permet :

une compréhension rapide

une maintenance facilitée

une collaboration entre testeurs

Chapitre 7

Conclusion

Ce projet a permis d'automatiser efficacement le test de la fonctionnalité Switch To Alert en appliquant toutes les exigences :

- automatisation complète via Selenium WebDriver,
- assertions strictes pour valider les résultats,
- captures d'écran automatiques en cas d'échec,
- génération d'un rapport d'exécution professionnel.

Le script est robuste, clair, modulaire et respecte les standards industriels du test automatisé.